

软件工程基础

第 4 章—软件项目计划

详细学习笔记

2026 年 5 月 28 日

一年之计在于春，一日之计在于寅

——《增广贤文》

中南大学计算机学院 任胜兵

摘要

本章系统介绍软件项目计划（**Software Project Planning**）的核心内容，包括：**风险分析（Risk Analysis）**——风险标识、风险估计、风险评价与风险管理监控；**进度安排（Scheduling）**——基本原则、工作量分配（40-20-40 规则）、PERT 技术与 Gantt 图方法；**项目组织（Project Organization）**——人员组织规律（Rayleigh-Norden 曲线、Brooks 定律）、人员组织形式（层次模式、矩阵模式、主程序员小组、民主小组、层次小组）；以及**软件项目开发计划文档（Software Project Development Plan Document）**的编写。

软件项目计划的**目标**是提供一个框架，使管理者能够对资源、成本、风险及进度进行合理的估算分析和调度，为软件工程过程提供管理依据。[p.3]

目录

1 软件项目计划概述 (Overview of Software Project Planning)	4
1.1 计划的重要性	4
1.2 为何需要计划	4
1.3 计划的内容框架	4
2 风险分析 (Risk Analysis ——4.1)	5
2.1 风险标识 (Risk Identification ——4.1.1)	5
2.1.1 风险的类型学分类	5
2.1.2 风险标识方法：风险项目检查表 (Bohem's Risk Checklist)	5
2.1.3 三类风险对照检查表	6
2.1.4 风险标识方法：头脑风暴法 (Brainstorming)	6
2.2 风险估计 (Risk Estimation ——4.1.2)	6
2.3 风险评价 (Risk Evaluation ——4.1.3)	7
2.3.1 风险影响类别示例	7
2.3.2 建立风险表 (Risk Table)	7
2.3.3 风险参考水准曲线 (Risk Referent Level Curve)	8
2.4 风险管理与监控 (Risk Management & Monitoring ——4.1.4)	8
2.4.1 风险管理策略	8
2.4.2 风险监控	9
3 进度安排 (Scheduling ——4.2)	9
3.1 进度安排的基本原则 (Basic Principles ——4.2.1)	9
3.2 工作量分配 (Work Distribution ——4.2.2)	9
3.2.1 40-20-40 规则 (40-20-40 Rule)	9
3.2.2 CAD 应用开发软件包工作量分配方案示例	10
3.3 进度安排方法 (Scheduling Methods ——4.2.3)	10
3.3.1 PERT 技术 (Program Evaluation & Review Technique)	10
3.3.2 计算事件的最早发生时刻 (Earliest Event Time, EET)	11
3.3.3 计算事件的最迟发生时刻 (Latest Event Time, LET)	11
3.3.4 确定关键路径 (Critical Path)	11
3.3.5 利用机动时间安排进度	11
3.3.6 Gantt 图方法 (Gantt Chart)	12
4 项目组织 (Project Organization ——4.3)	12
4.1 人员组织规律 (Personnel Organization Patterns ——4.3.1)	12
4.1.1 Rayleigh-Norden 曲线	12
4.1.2 各类人员参与程度曲线	13
4.1.3 人员-时间权衡定律 (Personnel-Time Tradeoff Law)	13
4.1.4 Brooks 定律 (Brooks' Law)	14

4.2 人员组织形式 (Organization Structures ——4.3.2)	14
4.2.1 层次模式 (Hierarchical Model)	14
4.2.2 矩阵模式 (Matrix Model)	15
4.2.3 主程序员小组 (Chief Programmer Team)	15
4.2.4 民主小组 (Democratic Team / Egoless Team)	15
4.2.5 层次小组 (Hierarchical Team)	16
5 软件项目开发计划文档 (Software Project Development Plan ——4.4)	16
5.1 目的与内容	16
5.2 计划文档的一般内容	17
6 小结 (Summary ——4.5)	17
附录：关键概念速查表	18

1 软件项目计划概述 (Overview of Software Project Planning)

1.1 计划的重要性

事实表明：软件工程项目的失败，大多是由于计划不周而引起的。[p.3] 这一观察强调了在软件工程实践中，项目计划处于核心地位。

软件项目计划的目标是提供一个**框架 (Framework)**，使管理者能够对**资源、成本、风险及进度**进行合理的估算分析和调度，为软件工程过程提供管理依据。[p.3]

软件项目计划一般由软件项目的**管理员、系统分析员与用户**共同制订。在需求分析阶段弄清软件系统的详细情况后才能正式定稿。[p.3]

1.2 为何需要计划

苏轼云：“谋于前才可不惑于后，临大事而不乱。” [p.4] 这句话精辟地概括了计划的根本价值。

计划的作用体现在四个方面：[p.4]

- (1) **风险**：计划有利于减少项目风险损失。通过预先识别和分析潜在风险，可以在风险发生前采取预防措施或制定应对方案。
- (2) **任务**：计划为项目管理提供了明确的目标。清晰定义的任务目标使团队知道努力方向，避免盲目工作。
- (3) **进度**：计划有利于高效利用资源，提高效益。通过合理的资源分配和时间安排，最大化资源利用率。
- (4) **组织**：计划有利于各成员相互配合，体现综合效应。明确的角色分工和沟通机制促进团队协作。

计划驱动的流程链为：[p.4]

计划 (Plan) $\Rightarrow$ 压力 (Pressure) $\Rightarrow$ 行动 (Action) $\Rightarrow$ 结果 (Result)

1.3 计划的内容框架

项目计划围绕三个核心维度展开：[p.5]

- **进度 (Schedule)** ——管理任务：确定各项任务的开始和结束时间，安排 workflow。
- **风险 (Risk)** ——管理失败：识别潜在风险，评估其影响，制定应对策略。
- **组织 (Organization)** ——管理机制：合理配置项目人员，建立有效的沟通和汇报结构。

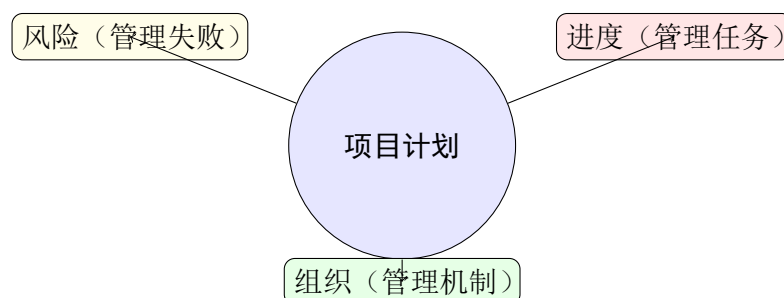


图 1: 项目计划的三个核心维度

2 风险分析 (Risk Analysis ——4.1)

T. Gilb 的名言：“如果你不主动攻击风险，风险就会主动攻击你。” [p.6]

主动的、明智的风险管理策略应该：在技术工作开始之前，先标识出潜在的风险，评估它们出现的概率及产生的影响，并按重要性加以排序，然后项目组织再制订一个计划来管理风险。 [p.6]

风险分析包含四项关键活动： [p.6]

- (1) 风险标识 (Risk Identification)
- (2) 风险估计 (Risk Estimation)
- (3) 风险评价 (Risk Evaluation)
- (4) 风险管理与监控 (Risk Management & Monitoring)

2.1 风险标识 (Risk Identification ——4.1.1)

2.1.1 风险的类型学分类

从宏观角度看，风险分为三类： [p.7]

- 项目风险 (Project Risk)：与项目预算、进度、人员等管理相关的风险。例如：资金不足（预算问题）。
- 技术风险 (Technical Risk)：与软件技术实现相关的风险。例如：规格说明存在二义性。
- 商业风险 (Business Risk)：与市场变化和产品价值相关的风险。例如：开发的产品过时。

从 Charette 的分类看，风险又可分为： [p.7]

- 已知风险 (Known Risk)：已经明确知道的风险。例如：用户计算机知识欠缺。
- 可预测风险 (Predictable Risk)：可以预见但发生概率不确定的风险。例如：用户改变需求。
- 不可预测风险 (Unpredictable Risk)：难以事先察觉的风险。例如：自然灾害。

2.1.2 风险标识方法：风险项目检查表 (Bohem's Risk Checklist)

Bohem 提出了系统的风险项目检查表，将风险按以下维度分类： [p.8]

- (1) 产品规模 (Product Scale)：与待开发或修改的软件的总体规模相关的风险。规模越大，管理复杂度和风险越高。
- (2) 商业影响 (Business Impact)：与管理或市场所加约束相关的风险。
- (3) 客户特性 (Customer Characteristics)：与客户的素质以及开发者和客户定期通信的能力相关的风险。
- (4) 过程定义 (Process Definition)：与软件过程被定义的程度以及它们被开发组织所遵守的程度相关的风险。
- (5) 开发环境 (Development Environment)：与用以建造产品的工具的可用性及质量相关的风险。
- (6) 建造技术 (Construction Technology)：与待开发软件的复杂性及系统所包含技术的“新奇性”相关的风险。

(7) **人员数量及经验 (Personnel Number & Experience)**: 与参与工作的软件技术人员的总体水平及项目经验相关的风险。

2.1.3 三类风险对照检查表

商业风险对照检查表 [p.9] 包含以下典型检查项:

- 产品是否满足市场需求?
- 是否存在竞争对手?
- 开发预算是否充足?
- 产品上市时间是否紧迫?
- 是否存在法律法规方面的约束?

技术风险对照检查表 [p.10] 包含以下典型检查项:

- 所用技术是否成熟?
- 技术方案是否经过验证?
- 是否需要开发新的算法或数据结构?
- 性能要求是否超出常规水平?
- 是否依赖第三方组件或接口?

管理风险对照检查表 [p.11] 包含以下典型检查项:

- 管理人员是否有足够的项目管理经验?
- 团队沟通是否顺畅?
- 是否存在人员流动风险?
- 开发进度是否过于紧张?
- 资源分配是否合理?

2.1.4 风险标识方法: 头脑风暴法 (Brainstorming)

头脑风暴法是风险标识的重要方法, 其流程如下: [p.12]

- (1) 将项目主要参与人员代表召集到一起。
- (2) 利用各自对项目不同部分的认知, 识别项目可能出现的问题。
- (3) 参会人员轮流发言, 无条件接纳任何意见, 不加评论。
- (4) 专人在白板上记录提出的每一条意见。
- (5) 轮流发言过程中, 成员可不发表意见而跳过。
- (6) 过程直到所有人员都没有新意见或限定时间到。

其他方法: Delphi 法 (德尔菲法)、事故树分析法 (Accident Tree Analysis)、因果图法 (Cause-Effect Diagram)、情景分析法 (Scenario Analysis) 等。 [p.12]

2.2 风险估计 (Risk Estimation —— 4.1.2)

风险估计包含两个核心维度的量化: [p.13]

- (1) **概率估计**: 估计风险发生的可能性。风险可能性尺度可以用布尔值 (Boolean)、定量 (Quantitative) 或定性 (Qualitative) 的方式表示。
- (2) **影响估计**: 估计与风险相关的问题出现后将会带来的损失。损失级别分为:

- 灾难的 (Catastrophic)
- 严重的 (Serious)
- 轻微的 (Minor)
- 可忽略的 (Negligible)

可能性等级示例 [p.14]: 风险发生的概率通常用一个数值范围或等级来描述, 例如:

- 极低 (Very Low): < 10%
- 低 (Low): 10% ~ 30%
- 中 (Medium): 30% ~ 50%
- 高 (High): 50% ~ 70%
- 极高 (Very High): > 70%

2.3 风险评价 (Risk Evaluation —— 4.1.3)

风险评价基于风险估计的结果, 建立一系列三维风险描述三元组 (Triple): [p.15]

$$[r_i, p_i, e_i]$$

其中:

- r_i : 表示第 i 个风险 (Risk)
- p_i : 表示该风险出现的概率 (Probability)
- e_i : 表示该风险产生的影响 (Effect)

评价过程包括以下步骤: [p.15]

- (1) 定义项目的各种**风险参考水准 (Risk Reference Levels)**, 如成本、进度等。
- (2) 找出每个三元组 $[r_i, p_i, e_i]$ 与各参考水准之间的关系。
- (3) 预测一组**临界点 (Threshold Points)** 以定义项目的**终止区 (Termination Region)**, 该区域由一条曲线或易变动区域来界定。
- (4) 预测怎样的风险组合, 会影响参考水准。

2.3.1 风险影响类别示例

风险产生的影响可以从多个维度来衡量 [p.16]:

- **性能 (Performance)**: 风险发生后对系统性能的退化程度
- **支持 (Support)**: 风险发生后对后续维护和支持工作的影响
- **成本 (Cost)**: 风险导致的额外费用
- **进度 (Schedule)**: 风险导致的进度延误程度

2.3.2 建立风险表 (Risk Table)

风险表的建立包括两个关键步骤: [p.17]

- (1) **估算风险概率值**: 可由开发团队各成员各自估算, 然后**投票 (Voting)**, 直到风险概率估算值接近为止。这是一种基于团队共识的概率确定方法。
- (2) **确定影响类别**: 针对四个影响因素 (性能、支持、成本、进度), 分别确定风险的影响类别, 然后将 4 个影响因素的影响类别求平均, 得到整体的影响类别。

表 1: 风险表样本 (Risk Table Sample) [p.18]

风险	概率	影响	优先级	缓解措施
需求变化	60%	高	1	建立变更控制流程
人员流失	20%	灾难	2	建立知识共享机制
技术不成熟	40%	严重	3	技术原型验证
性能不达标	15%	严重	4	早期性能测试

2.3.3 风险参考水准曲线 (Risk Referent Level Curve)

风险参考水准曲线用于确定项目的终止条件 [p.19]:

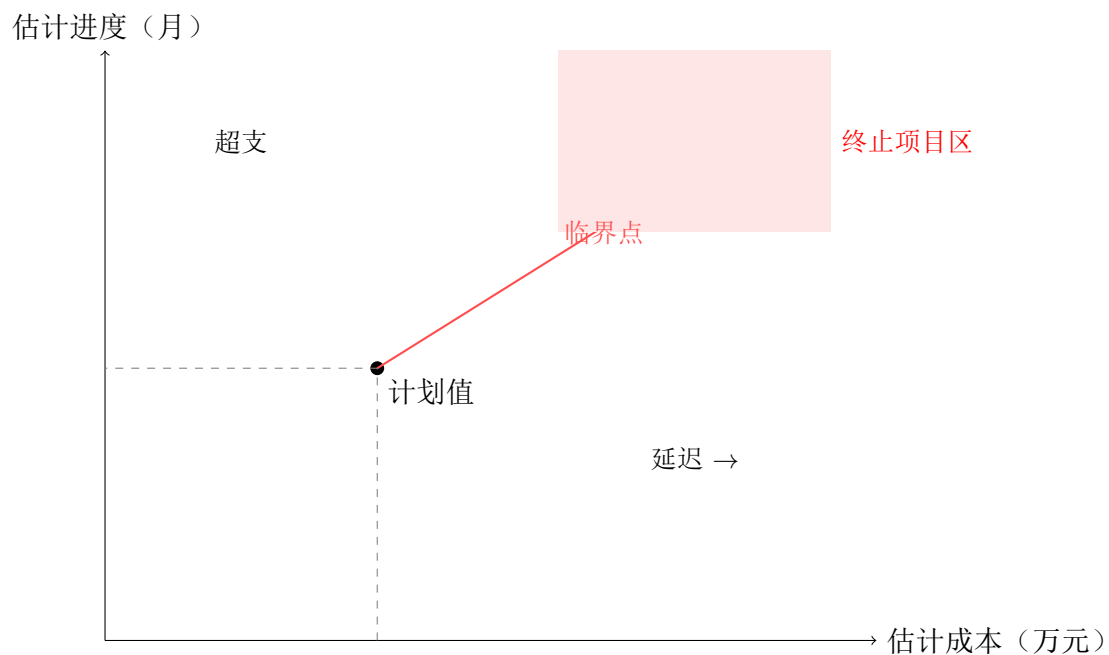


图 2: 风险参考水准曲线示意图

2.4 风险管理与监控 (Risk Management & Monitoring ——4.1.4)

2.4.1 风险管理策略

风险管理是指利用某些技术，如原型化 (Prototyping)、软件自动化 (Software Automation)、可靠性工程学 (Reliability Engineering)，以及某些项目管理方法等设法避免或转移风险。[p.20]

成本-效益权衡：实施风险管理策略会带来一些额外的开销。仅当实施风险管理策略所需的成本小于风险管理带来的效益（即风险带来的影响）时，才可考虑实施风险管理策略。[p.20]

优先级排序：[p.20]

- 高影响且发生概率为中到高的风险，以及低影响且高发生概率的风险，应该首先列入管理的考虑之中。
- 按照 Pareto 的 80-20 规则 (Pareto's 80-20 Rule)：80% 的软件风险能够由仅仅 20% 的已标出风险来说明。这意味着集中精力处理最重要的少数风险可以获得最大的收益。

2.4.2 风险监控

风险监控包含三个方面的持续工作：[p.21]

- (1) 事件和主要风险因素的跟踪：判断一个预测的风险事实上是否发生了。
- (2) 风险控制：确保针对某个风险制定的风险管理措施正在实施。
- (3) 信息收集：收集可用于将来风险分析的信息，为后续项目积累经验。

3 进度安排 (Scheduling ——4.2)

进度安排有两种基本方式：[p.22]

方式① 系统最终交付日期已经确定：软件开发组织在这一约束下将工作量进行分配。这是一种自顶向下（**Top-Down**）的安排方式，团队必须在固定截止日期前完成任务。

方式② 系统最终交付日期只确定了大致的期限：最终发布日期由软件开发组织确定，工作量以一种能够最好地利用资源的方式进行分配。这是一种更灵活的安排方式。

在实际工作中，第一种方式发生的频率远高于第二种方式。[p.22] 这意味着大多数软件项目都面临严格的交付时间约束。

3.1 进度安排的基本原则 (Basic Principles ——4.2.1)

进度安排需遵循以下七项基本原则：[p.23]

- (1) 任务分解（**Task Decomposition**）：将软件工程项目的任务分解成易管理的子任务，即作业（**Job**）。
- (2) 作业依存（**Job Dependency**）：确保作业间的依存关系，包括顺序关系（**Sequential**）和并发关系（**Concurrent**）。
- (3) 时间分配（**Time Allocation**）：为每个作业指定开始和终止时间。
- (4) 资源约束（**Resource Constraints**）：在进行时间分配时应考虑资源约束，如人员数量、工具等。
- (5) 定义责任（**Responsibility Definition**）：应指定某特定小组或个人负责某个作业。
- (6) 定义结果（**Deliverable Definition**）：对每个作业定义相应的结果，即产品或产品的一部分。
- (7) 定义里程碑（**Milestone Definition**）：每个作业或作业系列应与项目的里程碑相联系。里程碑是检查项目进展的关键节点。

3.2 工作量分配 (Work Distribution ——4.2.2)

3.2.1 40-20-40 规则 (40-20-40 Rule)

软件开发工作量的经典分配规则为 **40-20-40 规则**：[p.24]

- 编码（**Coding**）的工作量约占 **20%**
- 编码前的工作（计划、需求分析、设计）占 **40%**
- 编码后的工作（测试等）也占 **40%**

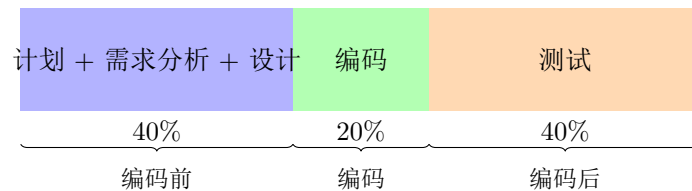


图 3: 40-20-40 工作量分配规则

实际统计数据显示，各阶段工作量分配大致为：[p.24]

- 计划（Planning）：约 2–3%
- 需求分析（Requirements Analysis）：约 10–15%
- 设计（Design）：约 20–25%
- 编码（Coding）：约 15–20%
- 测试（Testing）：约 30–40%

3.2.2 CAD 应用开发软件包工作量分配方案示例

对于 CAD（计算机辅助设计）应用开发软件包，典型的工作量分配方案包括以下阶段 [p.25]：

表 2: CAD 应用开发软件包工作量分配方案示例

阶段	工作量（人月）	占比
需求分析	4	13%
概要设计	5	17%
详细设计	6	20%
编码与单元测试	6	20%
集成测试	5	17%
确认测试	4	13%

3.3 进度安排方法 (Scheduling Methods ——4.2.3)

原则上可以把一般工程项目的进度安排方法和工具应用于软件工程项目。[p.26]

基本流程：[p.26]

- (1) 识别一组项目任务（作业）
- (2) 建立任务作业之间的相互关联
- (3) 估算各个任务的工作量
- (4) 分配人力和其它资源
- (5) 指定进度时序

两种核心方法：**PERT** 技术和 **Gantt** 图方法。[p.26]

3.3.1 PERT 技术 (Program Evaluation & Review Technique)

PERT (Program Evaluation & Review Technique, 计划评审技术)，又叫工程网络技术。[p.27]

历史背景：20 世纪 50 年代后期，美国海军和洛克希德公司（Lockheed）首次提出这一技术，并成功地应用于北极星导弹（Polaris Missile）的研究和开发。[p.27]

PERT 图的基本构成元素：[p.28]

- **事件（Event）：**项目中某个重要时刻，用节点（圆圈）表示
- **作业（Activity）：**从一个事件到另一个事件所需完成的工作，用有向箭头表示
- **依赖关系（Dependency）：**作业之间的先后顺序约束

PERT 图示例结构 [p.28]：一个典型的软件项目 PERT 网络包含以下事件和作业：

- 事件 1：项目开始 → 事件 2：需求分析（作业：分析，3 天）
- 事件 2 → 事件 3：概要设计（作业：概要设计，2 天）
- 事件 3 → 事件 4：详细设计（作业：详细设计，5 天）
- 事件 3 → 事件 5：测试计划（作业：测试计划，2 天）
- 事件 4 → 事件 6：编码（作业：编码，3 天）
- 事件 5 → 事件 6：测试方案设计（作业：测试方案设计，4 天）
- 事件 6 → 事件 7：产品测试（作业：产品测试，3 天）
- 事件 7 → 事件 8：文档整理（作业：文档整理，3 天）

3.3.2 计算事件的最早发生时刻 (Earliest Event Time, EET)

计算 EET 的规则：[p.29]

- (1) 通常规定，起始事件的最早发生时刻为 0。
- (2) 考虑进入该事件的所有作业。
- (3) 对于每个作业均计算其起始事件的最早发生时刻（EET）与持续时间之和。
- (4) 选取上述和数中的最大值作为该事件的最早发生时刻 EET。

3.3.3 计算事件的最迟发生时刻 (Latest Event Time, LET)

计算 LET 的规则：[p.30]

- (1) 通常规定，工程最后一个事件的最迟发生时刻等于最早发生时刻。
- (2) 考虑离开该事件的所有作业。
- (3) 对于每个作业都计算其结束事件的最迟发生时刻（LET）与作业持续时间之差。
- (4) 选取上述差数中的最小值作为该事件的最迟发生时刻 LET。

3.3.4 确定关键路径 (Critical Path)

关键事件（Critical Event）：事件的 EET 等于 LET 的事件。[p.31]

关键路径（Critical Path）：由关键事件组成的路径，在 PERT 图中通常用粗线箭头标示。
[p.31]

关键路径的意义：关键路径上的任何延迟都会直接导致整个项目的延迟。因此，项目经理需要重点关注关键路径上的作业。

3.3.5 利用机动时间安排进度

机动时间（Slack Time / Float Time）的计算公式： [p.32]

$$\text{机动时间} = \text{LET} - \text{EET} - \text{作业持续时间}$$

机动时间表示某个作业可以延迟而不影响项目总工期的最大时间量。进度安排策略为: [p.32]

- 先安排关键作业（机动时间为 0 的作业）
- 再利用机动时间安排非关键作业

3.3.6 Gantt 图方法 (Gantt Chart)

Gantt 图（甘特图）是另一种广泛使用的进度安排工具 [p.33]。与 PERT 图的网络结构不同，Gantt 图以时间轴为横坐标，以任务列表为纵坐标，用横条表示各项任务的开始时间、结束时间和持续时间。

Gantt 图的优点：

- 直观易懂，易于沟通
- 清楚显示任务的起止时间和进度
- 便于跟踪实际进展与计划的偏差

Gantt 图的局限：

- 不直接显示任务之间的依赖关系
- 难以识别关键路径
- 对于复杂项目，图可能过于庞大

实际应用：通常将 PERT 图和 Gantt 图结合使用——用 PERT 图分析任务依赖关系和关键路径，用 Gantt 图进行日常进度跟踪和汇报。

4 项目组织 (Project Organization ——4.3)

如何合理地组织参与项目的各类人员，以最大限度地发挥每个人的作用，对于成功地完成工程项目是非常必要的。[p.37]

D. L. Demarco 指出：软件项目中对于人员的管理问题不能像其它事件那样简单地划分，机械地对待。[p.37] 软件项目的人员管理具有高度的复杂性和特殊性。

4.1 人员组织规律 (Personnel Organization Patterns ——4.3.1)

4.1.1 Rayleigh-Norden 曲线

Rayleigh-Norden 曲线描述了软件开发项目中人力投入随时间的理想分布规律。[p.38]

如果在软件生存周期平均使用人力，则会造成以下问题：[p.38]

- 1) 起始阶段人力过剩：项目初期不需要大量人员，人员闲置造成浪费。
- 2) 开发后期和维护前期人力不足：开发和测试阶段需要大量人力时，人员不够导致进度延误。
- 3) 维护时期人力补偿为时已晚：到维护阶段再补充人员已无法弥补前期的延误。

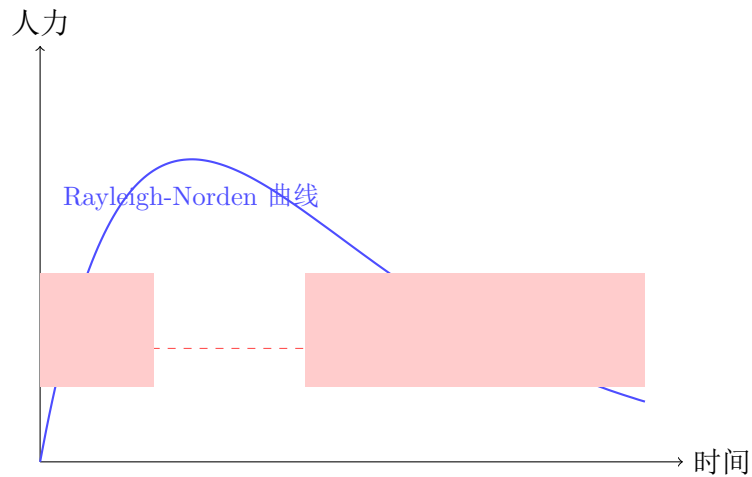


图 4: Rayleigh-Norden 曲线与平均人力分配的对比

4.1.2 各类人员参与程度曲线

不同角色的人员在软件项目各阶段的参与程度是不同的 [p.39]:

- **初级技术人员 (Junior Technical Staff):** 在编码时参与最多，而在其它阶段参与则较少。
- **高级技术人员 (Senior Technical Staff):** 在软件的开始阶段（需求分析、架构设计）和结束阶段（测试、部署）参与较多，在中间阶段参与较少。
- **管理人员 (Management):** 在项目开始阶段（计划、组织）参与较多，其它阶段参与较少。

表 3: 各类人员在不同阶段的参与程度

人员类型	计划	需求/设计	编码	测试	部署
高级技术人员	高	高	中低	高	高
初级技术人员	低	低	高	中	低
管理人员	高	中	低	低	低

4.1.3 人员-时间权衡定律 (Personnel-Time Tradeoff Law)

软件开发的工作量与时间之间存在重要的权衡关系: [p.40]

$$E \propto \frac{1}{t^4}$$

即软件开发工作量 (Effort) 与软件开发时间 (Time) 的 4 次方成反比。[p.40]

推论: [p.40]

- 如果适当延长软件开发时间，则可减少软件开发的工作量。因为团队有更充裕的时间进行设计、编码和测试，减少返工。
- 如果缩短软件开发时间，则工作量会大幅度增加。因为压缩进度导致沟通开销增加、质量下降和大量返工。

4.1.4 Brooks 定律 (Brooks' Law)

Brooks 定律 (Brooks' Law) 是软件工程领域最著名的经验法则之一: [p.41]

向一个已经延期的软件项目追加开发人员, 可能会使它完成得更晚。

解释: [p.41]

- 当开发人员以算术级数 (Arithmetic Progression) 增长时, 人员之间的通信 (Communication) 将以几何级数 (Geometric Progression) 增长。
- 新加入的人员需要时间学习和熟悉项目, 同时会占用老成员的时间进行指导和沟通。
- n 个人之间的通信路径数为 $\frac{n(n-1)}{2}$, 随人员数量呈平方级增长。

综合结论: 由人员-时间权衡定律和 Brooks 定律, 可以得出: [p.41]

对于软件项目, 开发时间宁可长一些, 开发人员可少而精一些。

4.2 人员组织形式 (Organization Structures —4.3.2)

软件开发机构选择怎样的人员组织形式, 要针对软件项目的特点和参与人员的素质来决定。
[p.42]

在建立软件开发组织时, 应注意三个原则: [p.42]

- (1) 责任到人: 尽早将责任落实, 便于管理。
- (2) 合理分工: 减少不必要的通信, 提高工作效率。
- (3) 责权均衡: 责任与权力的平衡, 有助于任务的完成。

4.2.1 层次模式 (Hierarchical Model)

层次模式是一种传统的组织结构 [p.43]:

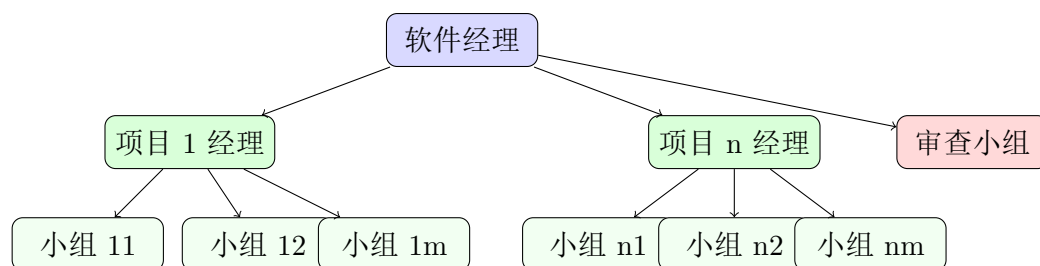


图 5: 层次模式组织结构

角色分工: [p.43]

- **软件经理 (Software Manager):** 负责整个开发部门的管理工作, 在各项目之间分配和协调各种资源。
- **项目经理 (Project Manager):** 负责一个具体项目开发的各个方面。
- **小组 (Team):** 每个小组负责项目的一部分工作。
- **审查小组 (Review Team):** 与项目经理同属一个层次, 主要从事质量保证活动, 在项目开发的里程碑进行技术审查和管理复审。

4.2.2 矩阵模式 (Matrix Model)

矩阵模式结合了项目管理和职能管理的优点 [p.44]:

表 4: 矩阵模式组织结构

	项目 1 经理	项目 2 经理	...	项目 n 经理
阶段 1 经理	子项 1,1	子项 2,1	...	子项 n,1
阶段 2 经理	子项 1,2	子项 2,2	...	子项 n,2
⋮	⋮	⋮	⋮	⋮
阶段 m 经理	子项 1,m	子项 2,m	...	子项 n,m

特点: [p.44]

- 项目经理 (Project Manager) 注重于问题域知识 (Problem Domain Knowledge) —— 即 “做什么”。
- 阶段经理 (Phase Manager) 则熟悉阶段的开发知识 (Phase-specific Development Knowledge) —— 即 “怎么做”。
- 每个成员同时向项目经理和阶段经理汇报, 形成双重汇报关系。

4.2.3 主程序员小组 (Chief Programmer Team)

主程序员小组最早由 H. Mills 提出, 并由 Baker 描述出来。[p.45]

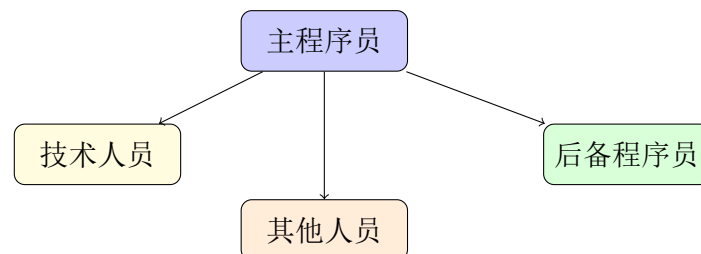


图 6: 主程序员小组结构

角色分工: [p.45]

- 主程序员 (Chief Programmer): 负责制订计划、协调与审查工作, 并完成结构设计及代码中关键和复杂部分的实现。
- 技术人员 (Technical Staff): 负责项目的具体分析与开发以及文档资料的编写工作。
- 后备程序员 (Backup Programmer): 主要协助主程序员的工作, 必要时可替代主程序员的工作。

历史实例: 1971 年, IBM 公司首次使用主程序员小组开发《纽约时报》的剪辑文件项目。[p.45]

4.2.4 民主小组 (Democratic Team / Egoless Team)

1971 年 G. M. Weinberg 首次描述了民主小组的组织形式。[p.46]

民主小组的最基本概念是“无我程序设计”（Egoless Programming）。[p.46] 其核心理念是：

- 代码不属于任何个人，而是属于整个团队。
- 任何人都可以审查和修改任何人的代码。
- 鼓励开放交流，消除个人防卫心理。
- 团队成员地位平等，决策通过协商达成。

文化背景：日本许多公司使用民主小组的组织形式进行软件开发，这与日本文化中强调集体协作的精神一致。[p.46]

4.2.5 层次小组 (Hierarchical Team)

层次小组结合了层次模式和民主模式的优点 [p.47]：

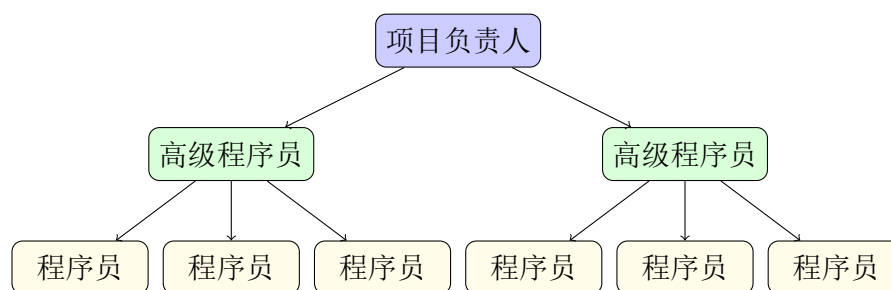


图 7: 层次小组结构

通信规则： [p.47]

- **层次间按隶属关系进行通信：**上下级之间通过正式渠道沟通。
- **组内平等通信：**同一层次内的成员可以自由平等地沟通。

适用场景：这种组织方式适合于大型软件开发项目，尤其是项目本身就是层次结构状的课题。[p.47]

5 软件项目开发计划文档 (Software Project Development Plan ——4.4)

5.1 目的与内容

软件开发计划文档的目的： [p.48]

用文件的形式，把对于在开发过程中各项目工作的负责人员、开发进度、所需经费预算、所需软/硬件条件等问题作出的安排记载下来，以便根据本计划开展和检查项目的开发工作。

重要说明： [p.48]

- (1) **只有在需求分析之后才能正式定稿：**需求分析确定了系统的详细功能和非功能需求后，才能制定准确的开发计划。

- (2) **可能要随项目的进展而不断改变**：软件开发计划是一个“活文档（Living Document）”，需要根据实际情况不断更新。

5.2 计划文档的一般内容

表 5: 软件项目开发计划文档的典型内容

章节	内容说明
1. 引言	项目背景、目标、范围、定义和参考资料
2. 项目估算	规模估算、工作量估算、成本估算、资源估算
3. 进度安排	任务分解、里程碑定义、PERT/Gantt 图
4. 风险管理	风险标识、风险估计、风险应对计划
5. 项目组织	团队结构、角色和职责、沟通计划
6. 质量计划	质量标准、评审计划、测试策略
7. 配置管理计划	版本控制、变更管理、配置审计
8. 工具与技术	开发工具、测试工具、管理工具

6 小结 (Summary ——4.5)

本章的核心内容涵盖三个主要方面：[p.49]

- (1) **风险分析 (Risk Analysis)**：通过风险标识、估计、评价和管理监控，能够做到“知己知彼”（彼即风险），从而“百战不殆”，使得开发者能够战胜风险带来的损失，使项目成功。
- (2) **进度安排 (Scheduling)**：进度安排的落空不仅会造成项目开发成本的提高，造成有形的经济损失，而且会使客户的不满意度和不信任度增加，造成无形的经济损失。
- (3) **项目组织 (Project Organization)**：在软件开发过程中，人是最活跃的部分。合理的人员组织和有效的团队管理是项目成功的根本保证。

附录：关键概念速查表

表 6: 第 4 章核心概念中英对照

中文术语	English Term
软件项目计划	Software Project Planning
风险标识	Risk Identification
风险估计	Risk Estimation
风险评价	Risk Evaluation
风险管理与监控	Risk Management & Monitoring
风险项目检查表	Risk Checklist
头脑风暴法	Brainstorming
三元组（风险描述）	Risk Triple $[r_i, p_i, e_i]$
风险参考水准	Risk Referent Level
终止区	Termination Region
80-20 规则	Pareto's 80-20 Rule
任务分解	Task Decomposition
作业依存	Job Dependency
里程碑	Milestone
40-20-40 规则	40-20-40 Rule
计划评审技术	PERT (Program Evaluation & Review Technique)
甘特图	Gantt Chart
最早发生时刻	EET (Earliest Event Time)
最迟发生时刻	LET (Latest Event Time)
关键路径	Critical Path
机动时间	Slack Time / Float Time
Rayleigh-Norden 曲线	Rayleigh-Norden Curve
人员-时间权衡定律	Personnel-Time Tradeoff Law
Brooks 定律	Brooks' Law
无我程序设计	Egoless Programming
主程序员小组	Chief Programmer Team
民主小组	Democratic Team
层次模式	Hierarchical Model
矩阵模式	Matrix Model

感谢各位聆听！祝大家学习愉快！

中南大学计算机学院 任胜兵